Problem Definition

$$D = A \times B \times C$$

Matrix multiplication between two matrices of dimensions N x N is of the order O(N^3). As N increases, so does the computation time. If the input size of matrices N_2 is 2N, then the computation time will be greater than 8 times the computation time taken for matrices of size N. In addition to the problem of high computation time, there are issues pertaining to real implementations due to hardware limitations, such as the retrieval time of data from memory and cache. Retrieving data from memory takes considerable time compared to ALU (Arithmetic Logic Unit) operations. While the presence of different levels of cache and registers can alleviate retrieval time on average, it does not fully resolve the issue.

In our particular problem, the matrix multiplication is between 3 different matrices A, B, and C of dimensions m x k, k x n, and n x l, respectively. The number of operations also depends on the order of the multiplication between A, B, and C.

Our main task is to improve the performance of the system by exploiting the underlying hardware capabilities of the cpu and memory system. We will use parallelism, loop reordering, and loop unrolling amongst other things to improve the performance of the program.

Improving Performance

D = A x B x C can be computed in two different ways: either by (A x B) x C or A x (B x C). In case AxB is computed first and then the result is multiplied with C, the number of operations will be

$$\text{Approximate total computations of order} = m.k.n + m.n.l = m.(k.n + n.l)$$

Otherwise, the number of operations will be

$$\text{Approximate total number of computations} = k.n.l + m.k.l = l.\ (m.k + n.k)$$

In order to potentially decrease the number of computations and improve the runtime, the first enhancement will be to select the order of multiplications (AxB)xC or Ax(BxC) based on the order that gives the minimum number of computations.

Instead of calculating m.(k.n + n.l) < l. (m.k + n.k) we use (k.n + n.l) / l < (m.k + n.k) due to more robustness from overflow which could lead to erroneous results for the inequality.

```
if (k.n + n.l) / l < (m.k + n.k) / m
        T ← AxB
        D ← TxC
else
        T ← BxC
        D ← AxT
```

Another enhancement is the order of the loops in the matrix multiplication. The order of loop multiplications can help in exploiting the underlying behavior of the memory system. We can exploit spatial locality by using the ikj order for the loops during matrix multiplication,

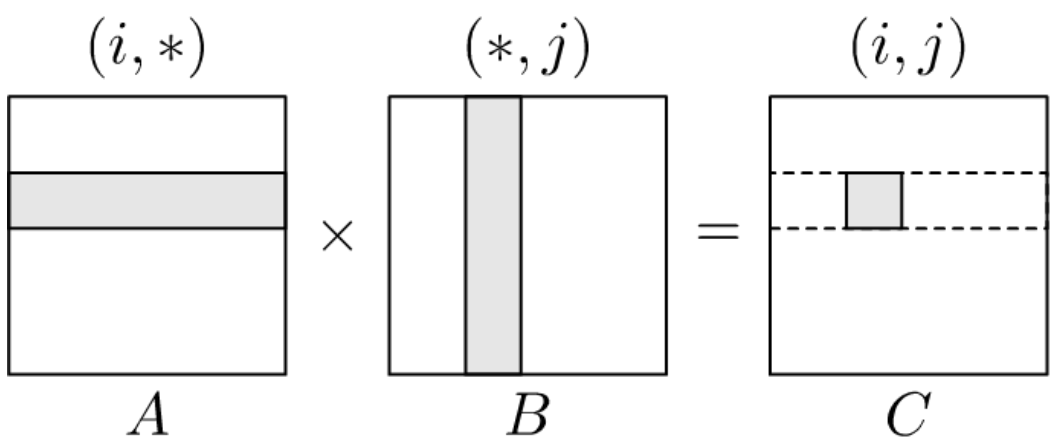


Fig. The ijk ordering for loops [1]

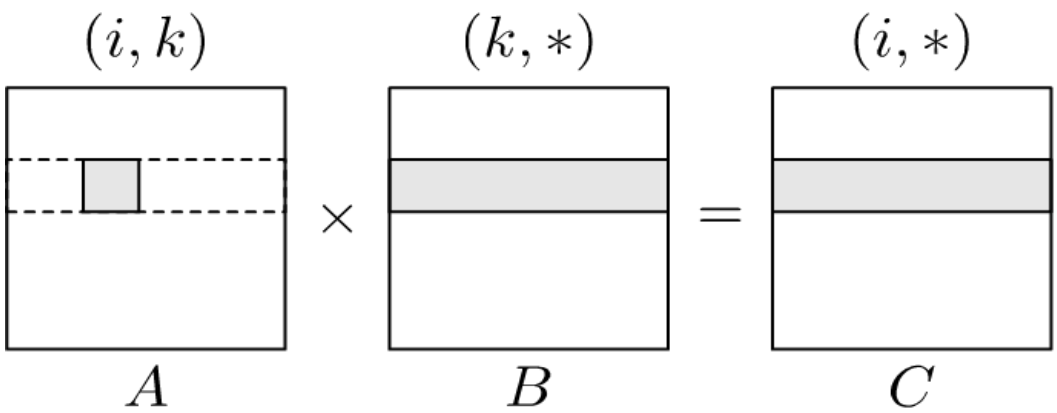


Fig. The ikj ordering for the loops which exploits spatial locality [1]

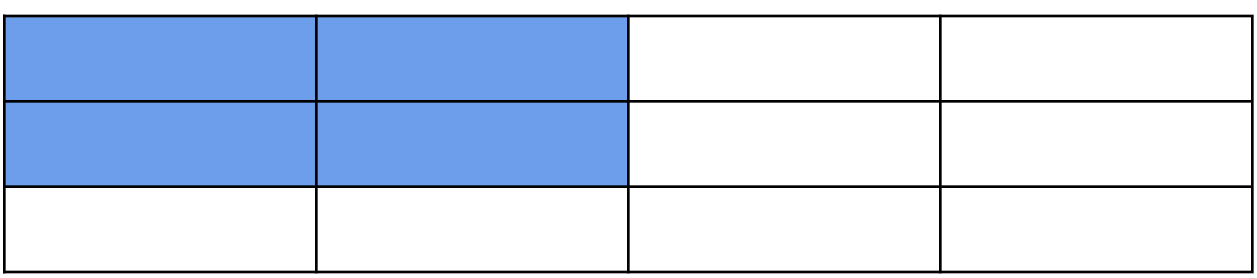
Parallelization

In order to implement the block partitioning for matrix multiplication we will block partitioning twice.

C = AxBxC

The first block partitioning will be for T, where T = A x B or B x C based on the criteria set above (See Improving Performance section). The T matrix (the intermediate matrix) will be divided into rectangles with dimensions p_0 and p_1 along the axis 0 (vertical or rows) and axis 1 (horizontal or column). The matrices, A and B or B and C, will also be partitioned in a similar manner to get the value of T. Let us assume T = A x B, then the partitioning will be such that we could calculate the different blocks of T independently. A will be divided in p_0 x k dimensions and B will be divided in k x p_1 dimensions.

If C is



A will be

and B will be

The program waits for the completion of evaluation of matrix T before the start of partitioning or evaluation of the D matrix. The intermediate result matrix will then be used to find the value of D; this will be achieved by partitioning D into blocks. The process for finding the value for different blocks of B will be similar.

Let b_0 and b_1 be the number of initial divisions of the rows and columns respectively, where b_0 * b_1 <= num_threads. The initial divisions ensure that (b_0 * b_1) / num_threads of the total rows and all columns will be covered.

b_0 ←floor of sqrt(num_threads)

B_1 ← floor of num_threads / b_0

Each of the blocks in this phase will be of dimensions

p_0 ← ((b_0 * b_1) / num_threads) * Rows / b_0

p_1 ← Columns / b_1

Let the remaining number of threads be r such that

r = num_threads - b_0 * b_1

We will repeat this process for the remaining number of threads ‘r’ until r becomes 0.

Some examples of the partitioning:

Example 1

If thread count is 4

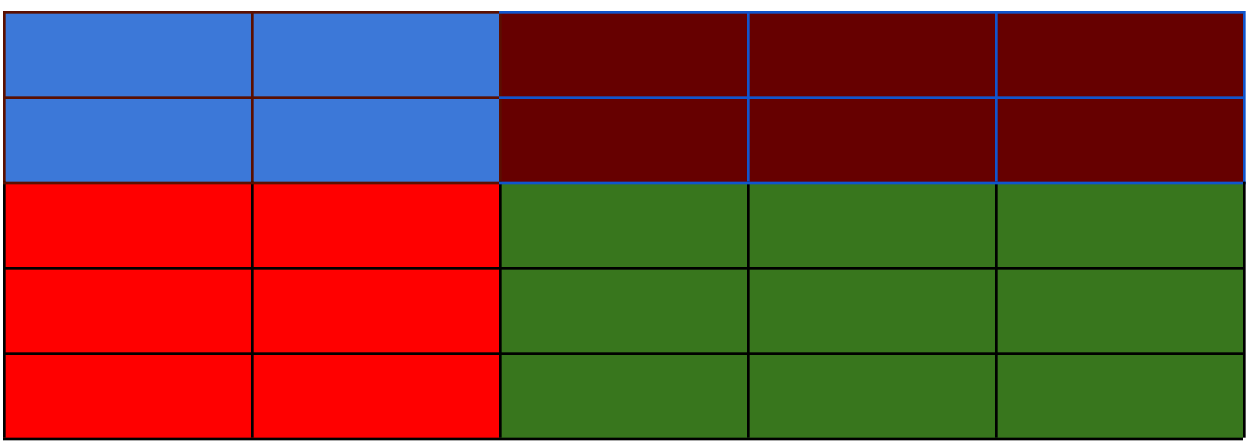


Fig. Colored sections represent different blocks of the matrix when dim = 5x5 and threads = 4.

B_0 = 2

B_1 = 2

r will be 0

Example 2

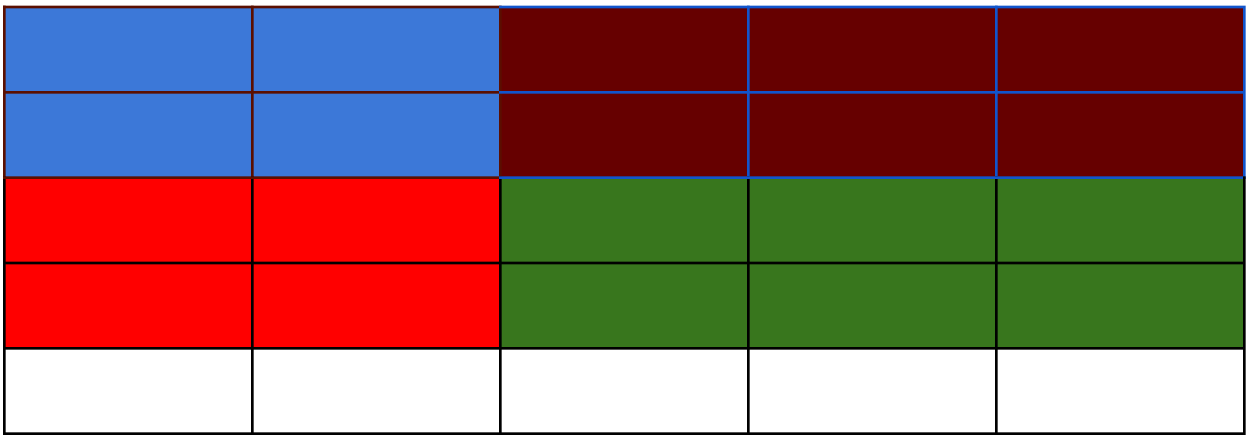


Fig. Intermediate colored blocks of the matrix when dim = 5x5 and threads = 5.

Final partition

B_0 = 2

B_1 = 2

r will be 1 > 0

So, after assigning 4 threads, another thread will be assigned to complete the white part.

If thread count is 5

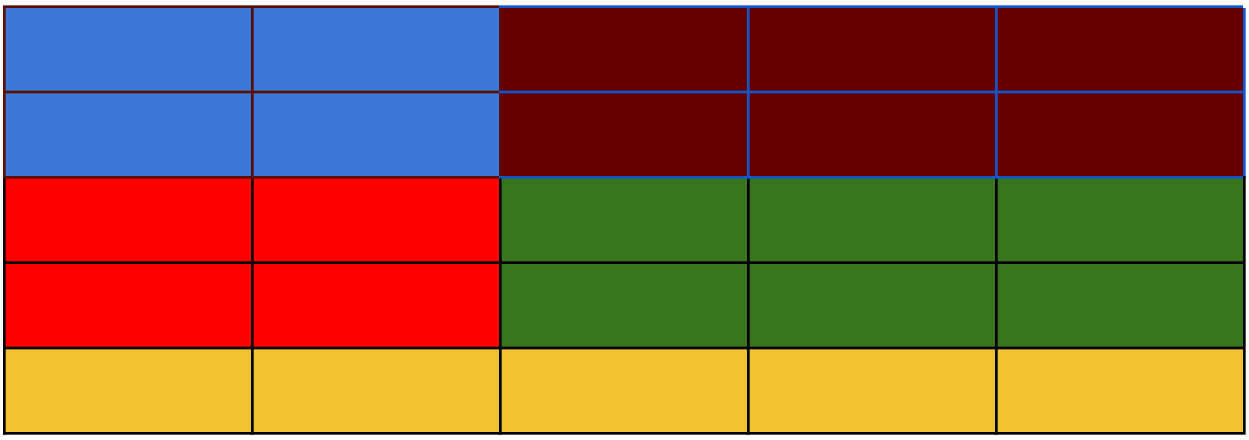


Fig. Final colored blocks of the matrix when dim = 5x5 and threads = 5.

Unrolling

In order to improve the performance of the algorithm via loop unrolling, we can use unrolling to improve the performance. The choice for unrolling can vary. There are three different ways we can do the unrolling. We experimented with two of the ways and found that unrolling j and k loops gave better and more consistent speedup. So, we unrolled along those axis:

```
int k = 0;
    while (k < (t->dims[1] - 4)) {
     int j = t->start[1];
     while (j < (t->dims[2] - 4)) {
      matrices_fl[2][i][j] += matrices_fl[0][i][k] * matrices_fl[1][k][j];
      matrices_fl[2][i][j] += matrices_fl[0][i][k+1] * matrices_fl[1][k+1][j];
      matrices_fl[2][i][j] += matrices_fl[0][i][k+2] * matrices_fl[1][k+2][j];
      matrices_fl[2][i][j] += matrices_fl[0][i][k+3] * matrices_fl[1][k+3][j];
      j++;
      matrices_fl[2][i][j] += matrices_fl[0][i][k] * matrices_fl[1][k][j];
      matrices_fl[2][i][j] += matrices_fl[0][i][k+1] * matrices_fl[1][k+1][j];
      matrices_fl[2][i][j] += matrices_fl[0][i][k+2] * matrices_fl[1][k+2][j];
      matrices_fl[2][i][j] += matrices_fl[0][i][k+3] * matrices_fl[1][k+3][j];
      j++;
      matrices_fl[2][i][j] += matrices_fl[0][i][k] * matrices_fl[1][k][j];
      matrices_fl[2][i][j] += matrices_fl[0][i][k+1] * matrices_fl[1][k+1][j];
      matrices_fl[2][i][j] += matrices_fl[0][i][k+2] * matrices_fl[1][k+2][j];
      matrices_fl[2][i][j] += matrices_fl[0][i][k+3] * matrices_fl[1][k+3][j];
      j++;
      matrices_fl[2][i][j] += matrices_fl[0][i][k] * matrices_fl[1][k][j];
      matrices_fl[2][i][j] += matrices_fl[0][i][k+1] * matrices_fl[1][k+1][j];
```

```
      matrices_fl[2][i][j] += matrices_fl[0][i][k+2] * matrices_fl[1][k+2][j];
      matrices_fl[2][i][j] += matrices_fl[0][i][k+3] * matrices_fl[1][k+3][j];
      j++;
    }
    while (j < t->dims[2]) {
      matrices_fl[2][i][j] += matrices_fl[0][i][k] * matrices_fl[1][k][j];
      matrices_fl[2][i][j] += matrices_fl[0][i][k+1] * matrices_fl[1][k+1][j];
      matrices_fl[2][i][j] += matrices_fl[0][i][k+2] * matrices_fl[1][k+2][j];
      matrices_fl[2][i][j] += matrices_fl[0][i][k+3] * matrices_fl[1][k+3][j];
      j++;
    }
    k+=4;
  }
  while (k < t->dims[1]) {
    int j = t->start[1];
    while (j < (t->dims[2] - 4)) {
      matrices_fl[2][i][j] += matrices_fl[0][i][k] * matrices_fl[1][k][j];
      matrices_fl[2][i][j+1] += matrices_fl[0][i][k] * matrices_fl[1][k][j+1];
      matrices_fl[2][i][j+2] += matrices_fl[0][i][k] * matrices_fl[1][k][j+2];
      matrices_fl[2][i][j+3] += matrices_fl[0][i][k] * matrices_fl[1][k][j+3];
      j+=4;
    }
    while (j < t->dims[2]) {
      matrices_fl[2][i][j] += matrices_fl[0][i][k] * matrices_fl[1][k][j];
      j++;
    }
    k++;
  }
```

Fig. Our preferred unrollong along the j and k dimensions

We unroll both j and k loops by a factor 4. The loops are also adjusted to ensure there is no out of bounds access or accidentally accessing other blocks which would cause erroneous results.

CPU core assignment

The main system that we used to test and parallelize our code on was the arm based M2 processor. On these systems, the operating system automatically assigns each of the threads a core, and we cannot directly assign a core to a thread (at least I couldn't find a way that worked). However, based on the performance improvement, the operating systems (MAC OS) automatically assigns each of the threads a core.

```
Model Name:            MacBook Pro
  Model Identifier:    Mac14,7
  Model Number:        Z16R0005SLL/A
  Chip:                Apple M2
  Total Number of Cores:     8 (4 performance and 4 efficiency)
  Memory:              16 GB
  Memory Bandwidth: 100 GB
```

We also tested our code on an intel i5 based computed with 4 cores with 2 hyperthreads each.The operating system used by the system was kali linux.

```
Command: lscpu
8 GB ram
Architecture:          x86_64
  CPU op-mode(s):      32-bit, 64-bit
  Address sizes:       39 bits physical, 48 bits virtual
  Byte Order:          Little Endian
CPU(s):                8
```

```
 On-line CPU(s) list:  0-7
Vendor ID:            GenuineIntel
 Model name:          Intel(R) Core(TM) i5-8265U CPU @ 1.60GHz
        CPU family:           6
        Model:        142
        Thread(s) per core:   2
        Core(s) per socket:   4

Additionally,
Memory Bandwidth: 30 GB
```

On linux systems, we used the following to assign each thread a core:

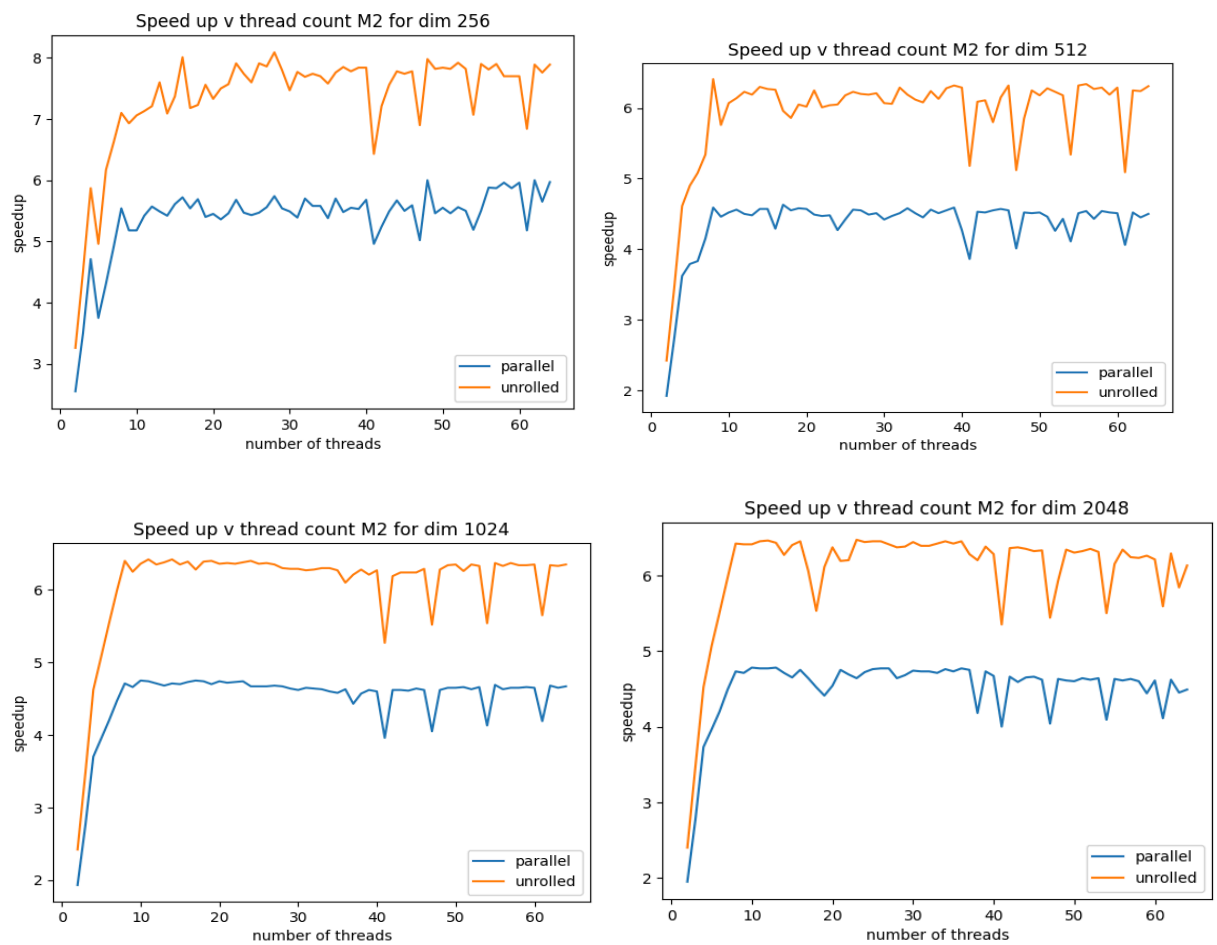
```
if (t->cpu_core != -1) {
        cpu_set_t cpuset;
        CPU_ZERO(&cpuset);
        CPU_SET(t->cpu_core % DEF_CORE, &cpuset);

        if (pthread_setaffinity_np(pthread_self(), sizeof(cpu_set_t), &cpuset) != 0) {
        perror("Pthread affinity error.");
        exit(EXIT_FAILURE);
        }
 }
```

Comparing the two processors, we can see that the M2 processor is better in all measures in comparison to the intel i5 processor. The number of cores provided on M2 is twice as many and each of the performance cores provide more than thrice FLOPS and around twice Memory Operations Per Second (MOPS). In addition to this, the memory bandwidth and RAM memory size is 3 times and 2 times higher for M2 in comparison to intel i5.

We were able to achieve a maximum speedup of 6.50 on the M2 processor. While on the intel i5 we could not achieve a speed up of more than 3.70. The change in the performance with increased matrix dimension sizes and number of threads was also visible; this is why we decided to mainly test the code on the M2 processor.

For upto 4 threads the performance on the PBS cluster was similar to that of intel i5 for medium sized arrays, so I decided to run the speed up tests on intel.

Performance Analysis



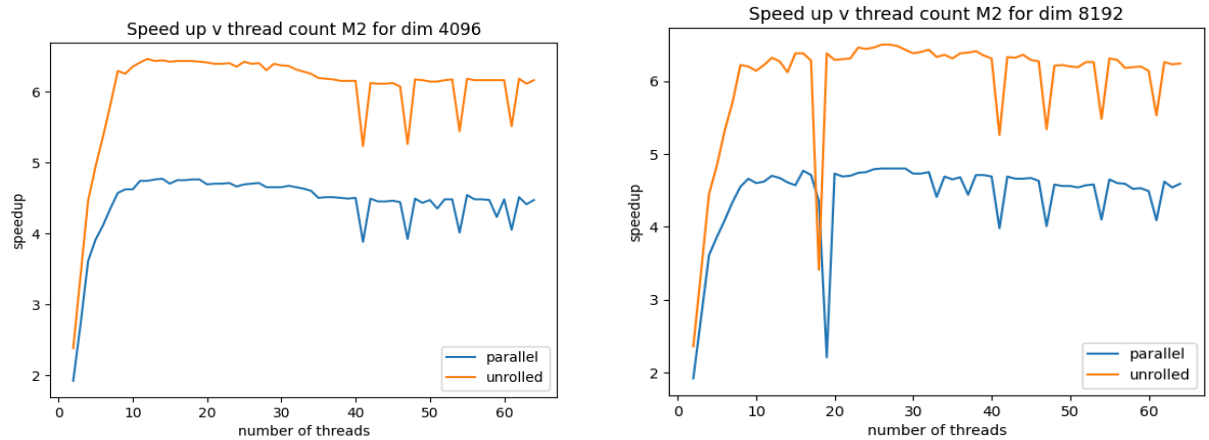


Fig. Performance/speedup of the parallelized program for different sizes of Matrix ranging from 2^8 to 2^13 for float type

Note: The sudden fall in the performance of the algorithm for dimension 8192 and 19 threads is due to sudden system shutdown and it does not have any other reason.

From the very start, we realized that the speedup of the algorithm was highly dependent on the underlying hardware. The M2 cpu system provided continuous and consistent scalability, while the intel i5 did not. A significant issue was also the runtime that the program took on each of the systems for large sized matrices (sizes greater than 2000 x 2000). The intel processor was significantly slower than the M2 for all cases; this shows the impact of the underlying hardware strength and architecture of both the systems on the performance of the parallelizable code.

One of the significant attributes about the performance on different systems is the fact that the performance of the parallelized systems becomes more consistent as the dimension of the matrix increases; this is due to lesser impact of the thread creation and parallelization overhead.

The peak performance of the program, in most cases, is reached when the number of threads is equal to the number of cores. In this case, there is no additional overhead caused due to scheduling. The decreased or less number of context switches necessary for this case also has an impact on the performance of the program. There are other influences, like the memory bandwidth and other system processes and applications that are running, on the performance of the parallel code which can compete for cpu cores and cause slowdown.

Cache thrashing is another issue that leads to decreased speedup in highly multithreaded programs; it could also be another contributing factor in slowdown of after a point where multiple threads are competing for the same cores.

An observation based on the difference between speedups of the unrolled+parallelized and the parallelized is that as the size of the of the matrices increases the difference between the increased speed up between the two goes from around 2 to 1.5 and stablizes around that point in this case. It can be seen as a result of diminishing or limited impact provided by the unrolling. As the sizes of the matrices increases it might be the case that memory latency, scheduling, and parallelism become the main influencing factor.

An issue with threads counts like 41, 47, and 61 et cetera might be due to the behavior of block partitioning algorithm.An explanation for this in the algorithm design., a thread is assigned a load that might create an imbalance in the load of the threads.The decline in the performance is visible in around these number of threads in all of the speedup visualizations.

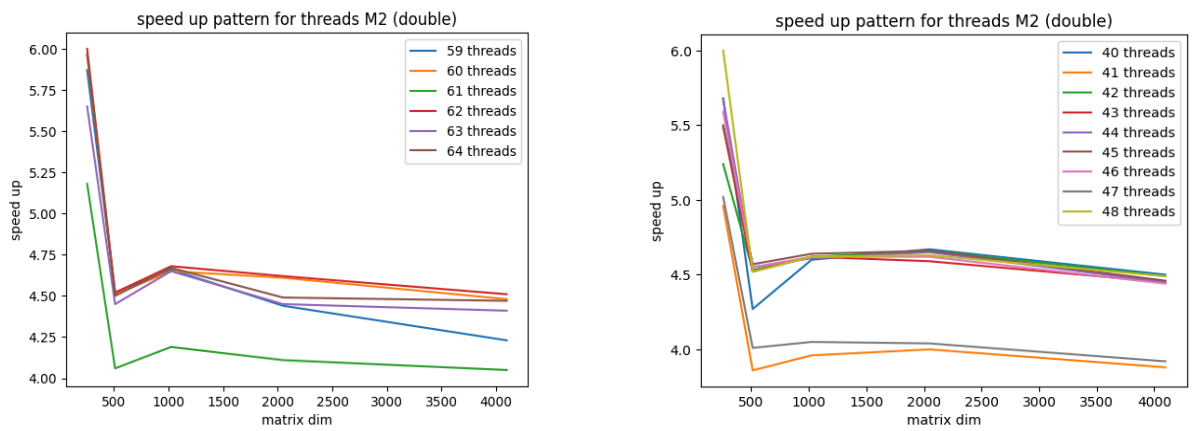


Fig. Program with 41, 47, and 61 threads have lower performance in their thread neighborhood

When the matrix size is smaller there is noise in the performance of the program that changes from run to run. The overhead caused by scheduling, parallelization, and synchronization can have impact on the performance of the program. This hypothesis is backed by the fact that the

changing performance starts after 8 threads (8 child threads + 1 main thread) when the speedup falls by around 0.6. It shows that the changing speedup is a result of underlying OS and hardware issues.

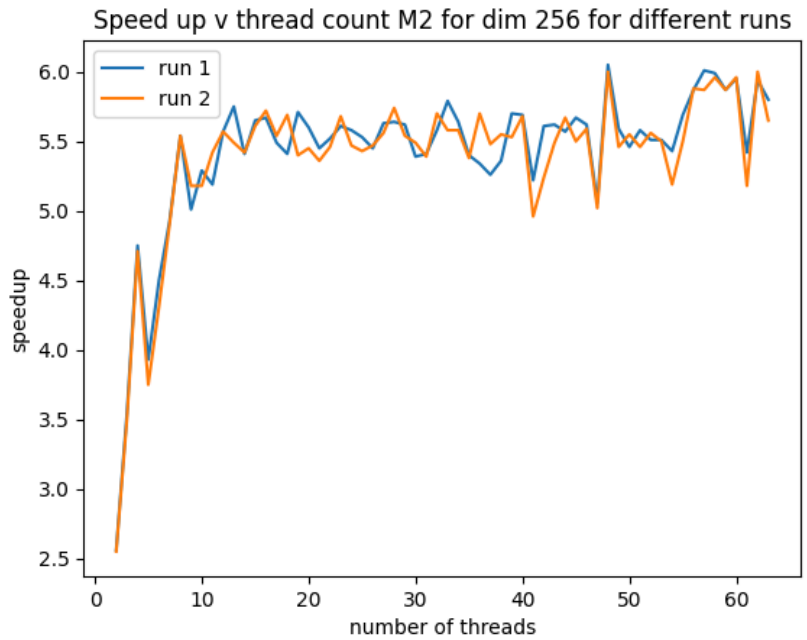


Fig. Performance change for a matrix with dimension = 256

To better understand the reason for this performance change after threads > total cores, we will look at the RAM stats for the program when the dimension of the matrices is 2048.

2048 2048 2048 2048 float 1

Time elapsed: 36.714409

2048 2048 2048 2048 float 8

Time elapsed: 7.827946

Threads:	9	Page Ins:	1
Ports:	19	Mach Messages In:	
CPU Time:	34,71	Mach Messages Out:	
Context Switches:	20918	Mach System Calls:	118
Faults:	6449	Unix System Calls:	193
Assertions:	0		

2048 2048 2048 2048 float 19

Time elapsed: 7.942664

Threads:	20	Page Ins:	1
Ports:	30	Mach Messages In:	
CPU Time:	35,03	Mach Messages Out:	
Context Switches:	28927	Mach System Calls:	172
Faults:	6476	Unix System Calls:	263
Assertions:	0		

2048 2048 2048 2048 float 41

Time elapsed: 9.271022

Threads:	40	Page Ins:	1
Ports:	50	Mach Messages In:	
CPU Time:	40,32	Mach Messages Out:	
Context Switches:	38645	Mach System Calls:	286
Faults:	6554	Unix System Calls:	399
Assertions:	0		

2048 2048 2048 2048 float 42

Time elapsed: 8.080595

Threads:	43	Page Ins:	1
Ports:	53	Mach Messages In:	
CPU Time:	41,95	Mach Messages Out:	
Context Switches:	38959	Mach System Calls:	288
Faults:	6550	Unix System Calls:	403
Assertions:	0		

We can see from the above RAM Statistics that when we run with 41 threads, there are only 40 threads that exist even though 41 are created in addition to the parent/main thread (checked during debugging) . Another important fact is that while the cpu time, context switches, and faults for both 41 thread and 42 theads are the same, the time taken by both 41 threads is less than 42 threads by about 1.1 seconds. This again points to a possible straggler thread.

Another important thing to note is that generally as the number of threads increases the number of context switches increases by almost 1.45 times. Additionally, the number of faults also

increases as the number of threads are increasing. The faults could be increasing due to cache thrashing, as a result of different memory accesses by different threads increasing.

Performance for double on M2

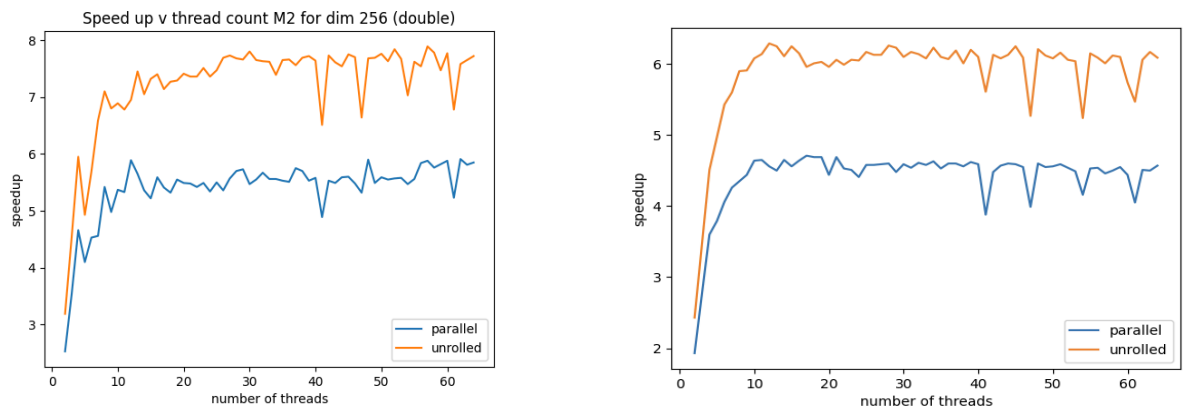


Fig. For sizes 256 and 512 respectively on M2

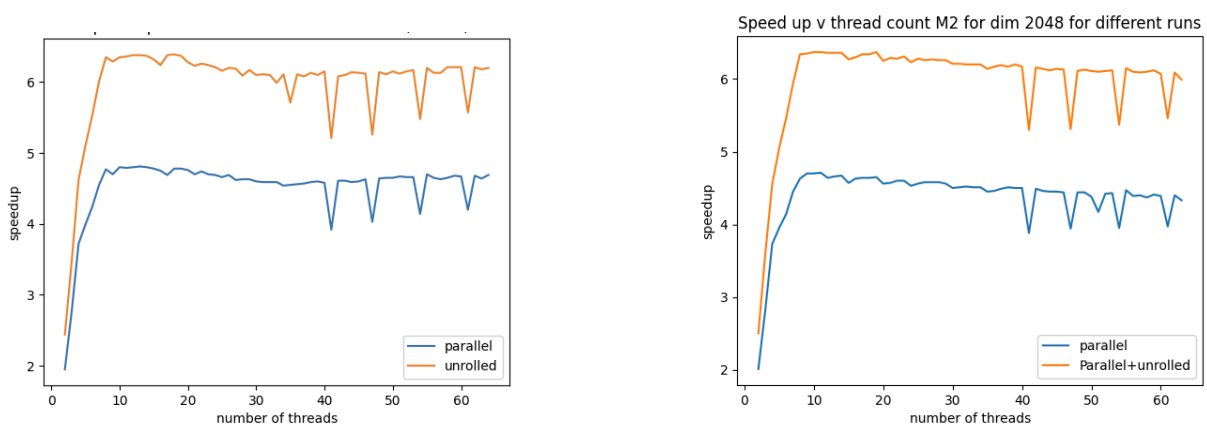


Fig. For sizes 1024 and 2048 on M2

./exe 2048 2048 2048 2048 double 1

Time elapsed: 37.041588

./exe 2048 2048 2048 2048 double 8

Time elapsed: 8.599001

Threads:	9	Page Ins:	1
Ports:	19	Mach Messages In:	
CPU Time:	53,82	Mach Messages Out:	
Context Switches:	32695	Mach System Calls:	126
Faults:	10571	Unix System Calls:	195
Assertions:	0		

./exe 2048 2048 2048 2048 double 19

Time elapsed: 8.163104

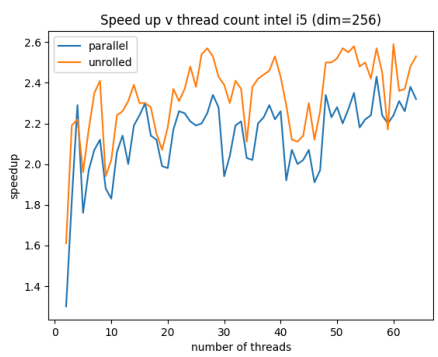
Threads:	20	Page Ins:	2
Ports:	30	Mach Messages In:	
CPU Time:	57,60	Mach Messages Out:	
Context Switches:	45922	Mach System Calls:	181
Faults:	10604	Unix System Calls:	265
Assertions:	0		

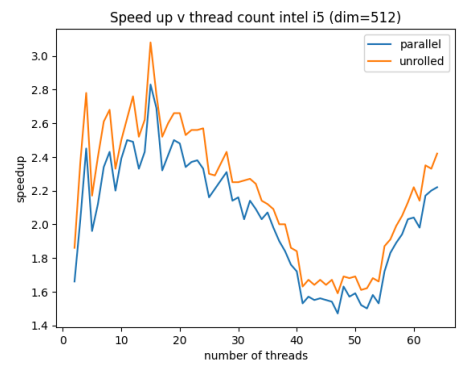
When we use double instead of float there is a general increase in the time that the algorithm takes to complete the same dimensional input. There are numerous reasons that this is happening. Some of them are:

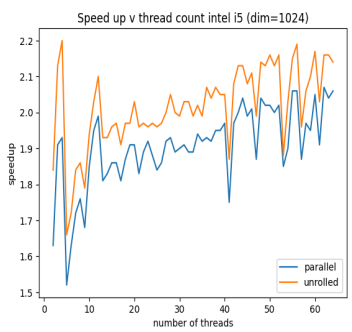
- The size of a double is usually 2 the size of a float. It is used to give higher precision representation of decimal numbers. So, it takes more storage to store the equivalent number of double precision points in C.
- It can take higher time to operate on double precision numbers as compared to single precision numbers. The difference can be more visible depending on the system in use.

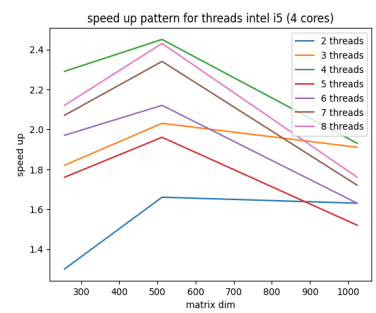
- Comparing the RAM statistics of double matrices in comparison to float matrices, we can see that the number of context switches and faults are also higher which leads to increased overhead.
- Another important aspect is that the number of page ins in the case of 19 threads is higher in the case of double which shows that there might be cases when the storage needs could lead to having to retrieve the data from the hard drive.

Performance on Intel i5



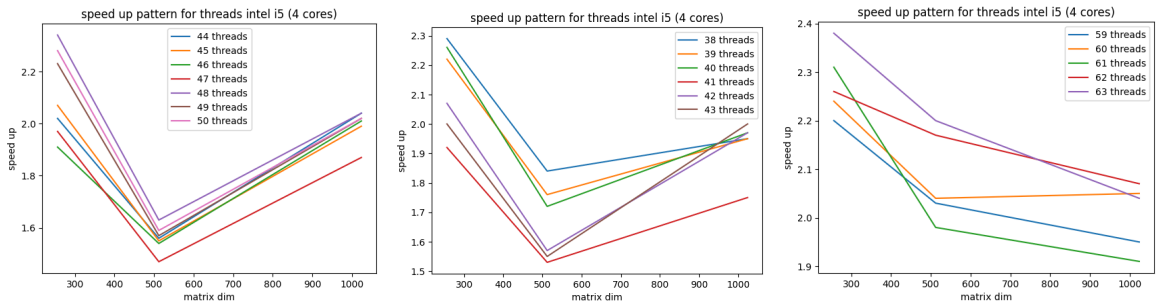






We can observe from the performance of the floating matrix multiplication on the intel i5 (4 cores) that the program performs the best on large scaled or medium scaled matrices when the number of threads is equal to the number of cores on the system (4 in this case). We can also see that the performance of the 4 threaded core is cosistently one of the best in all sizes.

We can also observe that the unrolling works properly and it gives higer speed up with the unrolling.



Another similar aspect is the fall in performance around thread count 41, 47 and 61; this shows that this issue is highly likely to be a result of an issue in the load allocation in the parallel algorithm designed. We can also observe that the unrolling definitely improve the performance of the algorithm on the different divices.

The peak performance in small dimensional matrices is a result of the same phenomenon as in the case of M2 processor: the smaller the size of the matrices the less cache thrashing and cache faults occur.

User Manual

1. Setting the Number of Cores

You can configure the number of cores the program utilizes in two ways:

- **Directly modifying the header file:** Open the matrix_mult.h file and locate the definition for DEF_CORE. Change its current value (which is 8) to the desired number of cores.
- **Using the set_total_cores() function:** You can programmatically set the number of cores by calling the set_total_cores(int i) function within your code, where i represents the desired number of cores.

2. Compiling and Running the Main Matrix Multiplication Code

To compile the main program, which performs the matrix multiplication, open your terminal and execute the following command:

```
gcc -pthread main.c matrix_mult.c -o exe -lm -Wall
```

This command will:

- gcc: Invoke the GNU Compiler Collection.
- -pthread: Link the POSIX Threads library, necessary for parallel execution.
- main.c and matrix_mult.c: Specify the source code files to be compiled.
- -o exe: Name the output executable file exe.
- -lm: Link the math library.
- -Wall: Enable all compiler warnings.

After successful compilation, you can run the exe file using the following syntax:

Bash
```
./exe m k l n float num_threads
```

Where:

- m: Number of rows in matrix A.
- k: Number of columns in matrix A and rows in matrix B.
- l: Number of columns in matrix B and rows in matrix C.
- n: Number of columns in matrix C.
- float: The data type used for the matrices (currently only 'float' is supported via command-line argument).
- num_threads: The number of threads to use for parallel execution.

Output:

The program will then execute the matrix multiplication (A x B x C = D) and print the time taken to complete the operation.

3. Compiling and Running for Speedup and Comparison

To compile a separate executable that calculates the speedup and compares the results of different multiplication methods, use the following command:

Bash
```
gcc -pthread sp.c matrix_mult.c -o speed -lm -Wall
```

Similar to the main compilation, this creates an executable named speed. You can run this executable with the same command-line arguments as the main program:

terminal
./speed m k l n float num_thread

Output:

This program will output the speedup achieved by the parallelized version compared to the serial version, as well as a comparison value between the parallelized and parallelized + unrolled implementations to check for correctness.

4. Code Structure and Function Usage

The program is structured with several functions to manage matrix operations. Here's how you can use them to create a custom matrix multiplication process:

1. **Set Data Type:** Call the set_type(int i) function to define the data type of the matrices.
    - i = 0: Sets the data type to float (default).
    - i = 1: Sets the data type to double.

```
set_type(0); // Set to float
set_type(1); // Set to double
```

2. **Allocate Matrices:** Use the allocate_array(int d1, int d2, int mode) function to allocate memory for a 2D matrix.
    - d1: Number of rows.
    - d2: Number of columns.
    - mode: Initialization mode.
        - mode = 0: Initialize the matrix with zeros.
        - mode = 1: Initialize the matrix with random values.
          void** matrix_A = allocate_array(m, k, 1); // Allocate matrix A with random values

```
void** matrix_B = allocate_array(k, l, 1); // Allocate matrix B with random values
void** matrix_C = allocate_array(l, n, 1); // Allocate matrix C with random values
void** matrix_D = allocate_array(m, n, 0); // Allocate matrix D initialized to zeros
```

3. **Set Matrices:** Populate the allocated matrices using the set_matrix(void** X, int i) function. This function likely assigns the allocated memory to the appropriate matrix pointer within the program's structure. Call this function for all four matrices (A, B, C, and D) in the correct order.
    - X: The pointer to the allocated matrix.
    - i: The index of the matrix:
        - i = 0: Matrix A
        - i = 1: Matrix B
        - i = 2: Matrix C
        - i = 3: Matrix D

```
set_matrix(matrix_A, 0);
set_matrix(matrix_B, 1);
set_matrix(matrix_C, 2);
set_matrix(matrix_D, 3);
```

4. **Set Dimensions and Threads:** Before running the matrix multiplication, set the dimensions of the matrices and the number of threads using the following functions:
    - set_dims(int m, int k, int l, int n): Sets the dimensions for matrices A (m x k), B (k x l), and C (l x n).
    - set_threads(int j): Sets the number of threads to be used for parallel execution.

```
set_dims(m_value, k_value, l_value, n_value);
set_threads(num_threads_value);
```

5.
6. **Perform Matrix Multiplication:** Call one of the following functions to perform the matrix multiplication:
    - void block2d_matmul(): Implements a 2D block matrix multiplication algorithm.
    - void serial_matmul(): Implements a standard serial matrix multiplication algorithm.
    - int unroll_matmult(): Implements a matrix multiplication algorithm with loop unrolling.

```
block2d_matmul();
// or
serial_matmul();
// or
```

```
unroll_matmult();
```

7. **Compare Results:** After performing the multiplication, you can compare the resulting matrix with an expected output using the compare(void** X, void** Y, int l, int m) function.
    - X: Pointer to the first matrix.
    - Y: Pointer to the second matrix.
    - l: Number of columns in the matrices being compared.
    - m: Number of rows in the matrices being compared.

```
// Assuming 'expected_D' is the expected result of the multiplication
int comparison_result = compare(matrix_D, expected_D, n_value, m_value);

if (comparison_result == 0) {
   printf("Matrices are identical.\n");
} else {
   printf("Matrices are different.\n");
}
```

8. **Deallocate Memory:** Once you are finished with the matrices, it is crucial to free the allocated memory using the delete_matrix(void** arr, int d1) function.
    - arr: Pointer to the matrix to be deallocated.
    - d1: Number of rows in the matrix.

```
delete_matrix(matrix_A, m_value);
delete_matrix(matrix_B, k_value);
delete_matrix(matrix_C, l_value);
delete_matrix(matrix_D, m_value);
```

9.

This manual should provide you with the necessary information to understand, compile, and utilize the matrix multiplication program. Remember to consult the source code for more detailed information on the implementation of each function.

Appendix A

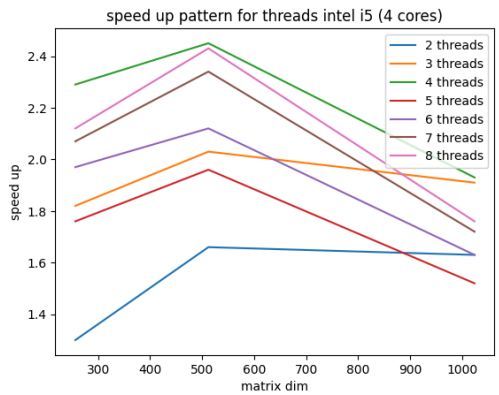
An alternative less efficient way of unrolling the loops

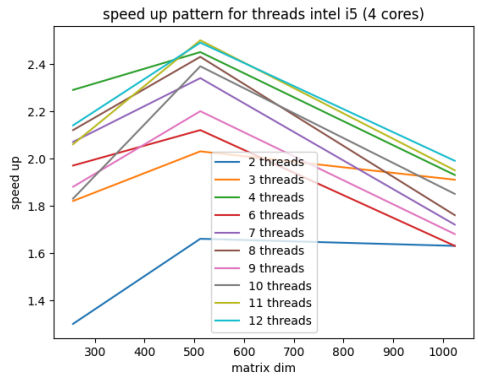
```
int i = t->start[0];
        while (i < (t->dims[0] - 4)) {

        int k = 0;
        while (k < (t->dims[1] - 4)) {

        int j = t->start[1];
        while (j < t->dims[2]) {
        matrices_db[2][i][j] += matrices_db[0][i][k] * matrices_db[1][k][j];
        matrices_db[2][i][j] += matrices_db[0][i][k+1] * matrices_db[1][k+1][j];
        matrices_db[2][i][j] += matrices_db[0][i][k+2] * matrices_db[1][k+2][j];
        matrices_db[2][i][j] += matrices_db[0][i][k+3] * matrices_db[1][k+3][j];
        i++;

        matrices_db[2][i][j] += matrices_db[0][i][k] * matrices_db[1][k][j];
        matrices_db[2][i][j] += matrices_db[0][i][k+1] * matrices_db[1][k+1][j];
        matrices_db[2][i][j] += matrices_db[0][i][k+2] * matrices_db[1][k+2][j];
        matrices_db[2][i][j] += matrices_db[0][i][k+3] * matrices_db[1][k+3][j];
        i++;

        matrices_db[2][i][j] += matrices_db[0][i][k] * matrices_db[1][k][j];
        matrices_db[2][i][j] += matrices_db[0][i][k+1] * matrices_db[1][k+1][j];
        matrices_db[2][i][j] += matrices_db[0][i][k+2] * matrices_db[1][k+2][j];
        matrices_db[2][i][j] += matrices_db[0][i][k+3] * matrices_db[1][k+3][j];
        i++;

        matrices_db[2][i][j] += matrices_db[0][i][k] * matrices_db[1][k][j];
        matrices_db[2][i][j] += matrices_db[0][i][k+1] * matrices_db[1][k+1][j];
        matrices_db[2][i][j] += matrices_db[0][i][k+2] * matrices_db[1][k+2][j];
        matrices_db[2][i][j] += matrices_db[0][i][k+3] * matrices_db[1][k+3][j];
        i-=3;
        j++;
        }

        k+=4;
        }

        while (k < t->dims[1]) {
        for (int j = 0; j < t->dims[2]; j++) {
```

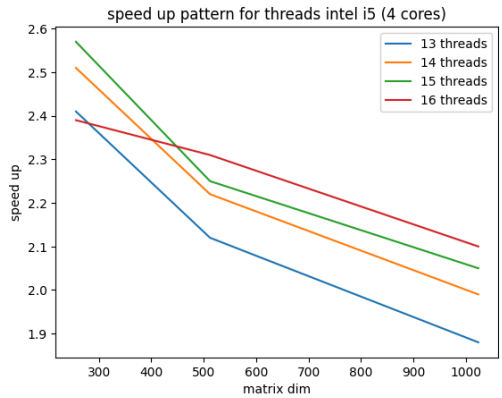
```
matrices_db[2][i][j] += matrices_db[0][i][k] * matrices_db[1][k][j];
matrices_db[2][i+1][j] += matrices_db[0][i+1][k] * matrices_db[1][k][j];
matrices_db[2][i+2][j] += matrices_db[0][i+2][k] * matrices_db[1][k][j];
matrices_db[2][i+3][j] += matrices_db[0][i+3][k] * matrices_db[1][k][j];
}
k++;
}

i += 4;
}

while (i < t->dims[0]) {
int k = 0;
while (k < (t->dims[1]-4)) {
for (int j = 0; j < t->dims[2]; j++) {
matrices_db[2][i][j] += matrices_db[0][i][k] * matrices_db[1][k][j];
matrices_db[2][i][j] += matrices_db[0][i][k+1] * matrices_db[1][k+1][j];
matrices_db[2][i][j] += matrices_db[0][i][k+2] * matrices_db[1][k+2][j];
matrices_db[2][i][j] += matrices_db[0][i][k+3] * matrices_db[1][k+3][j];
}

k+=4;
}

while (k < t->dims[1]) {
for (int j = 0; j < t->dims[2]; j++) {
matrices_db[2][i][j] += matrices_db[0][i][k] * matrices_db[1][k][j];
}
k++;
}

i++;
}
```

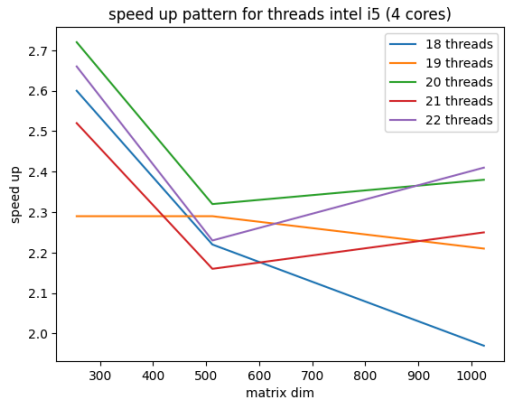
Appendix B

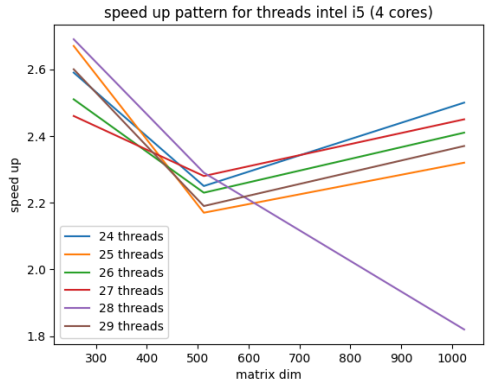
More speedup graphs for intel run

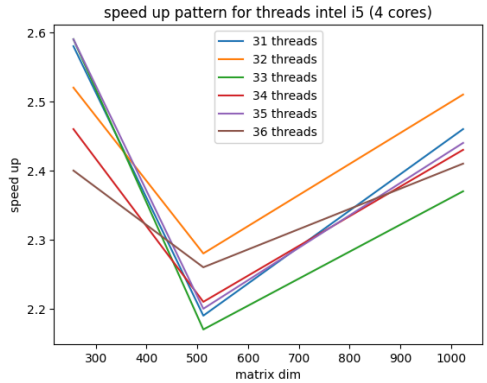

speed up pattern for threads intel i5 (4 cores)
2 threads
3 threads
4 threads
5 threads
6 threads
7 threads
8 threads
speed up
matrix dim

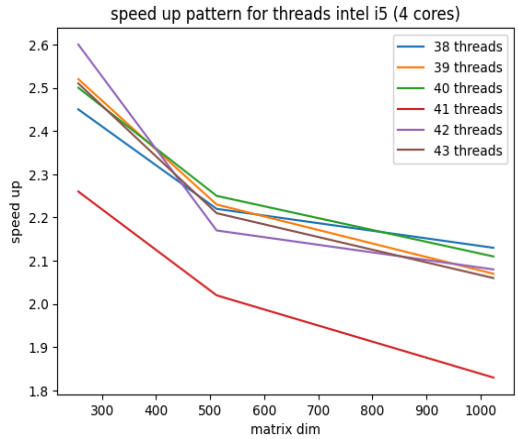

speed up pattern for threads intel i5 (4 cores)
2 threads
3 threads
4 threads
6 threads
7 threads
8 threads
9 threads
10 threads
11 threads
12 threads
speed up
matrix dim

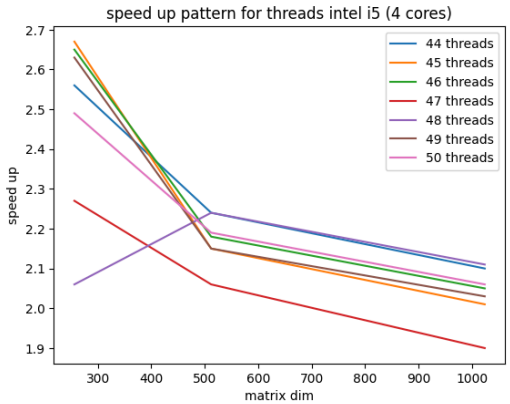

speed up pattern for threads intel i5 (4 cores)
13 threads
14 threads
15 threads
16 threads
speed up
matrix dim

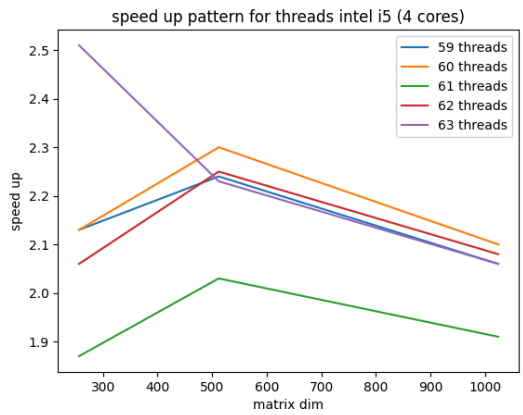

speed up pattern for threads intel i5 (4 cores)
18 threads
19 threads
20 threads
21 threads
22 threads
speed up
matrix dim


speed up pattern for threads intel i5 (4 cores)
24 threads
25 threads
26 threads
27 threads
28 threads
29 threads
speed up
matrix dim

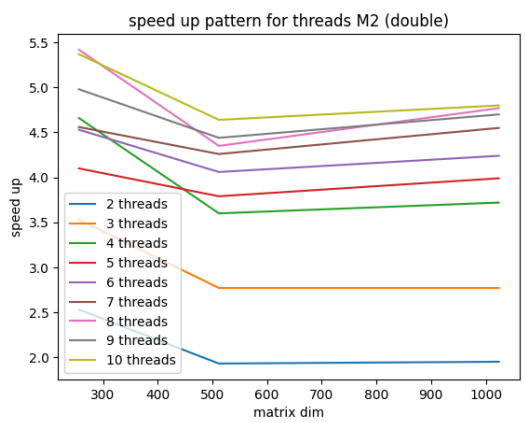

speed up pattern for threads intel i5 (4 cores)
31 threads
32 threads
33 threads
34 threads
35 threads
36 threads
speed up
matrix dim

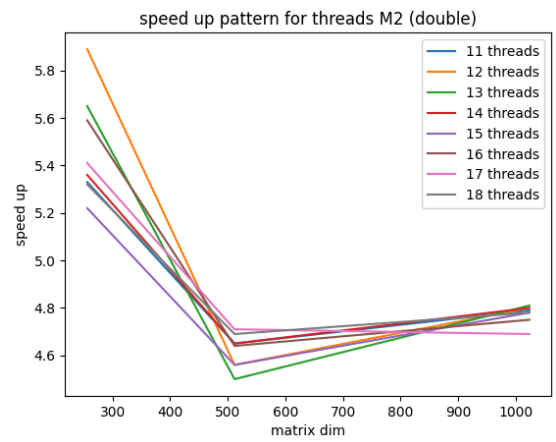

speed up pattern for threads intel i5 (4 cores)
38 threads
39 threads
40 threads
41 threads
42 threads
43 threads
speed up
matrix dim

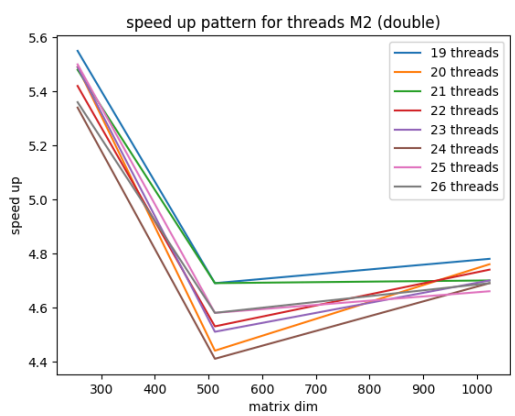

speed up pattern for threads intel i5 (4 cores)
44 threads
45 threads
46 threads
47 threads
48 threads
49 threads
50 threads
speed up
matrix dim

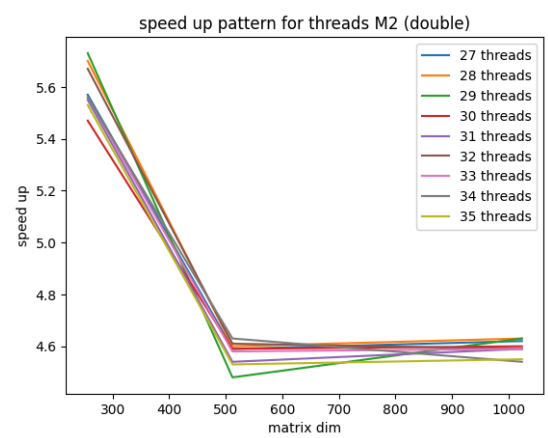


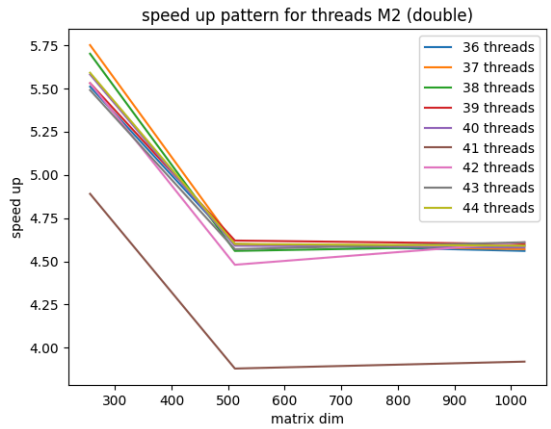
Appendix C
M2 double performance for just parallelized (not unrolled)

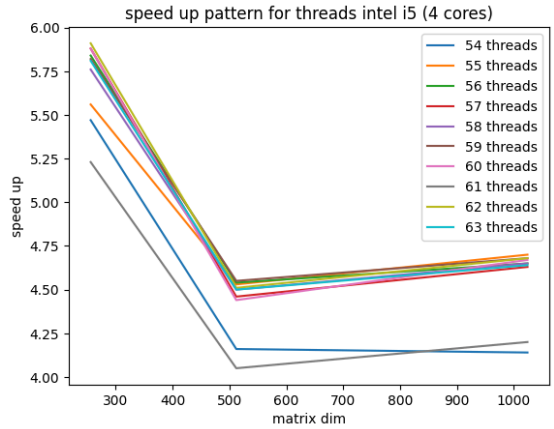


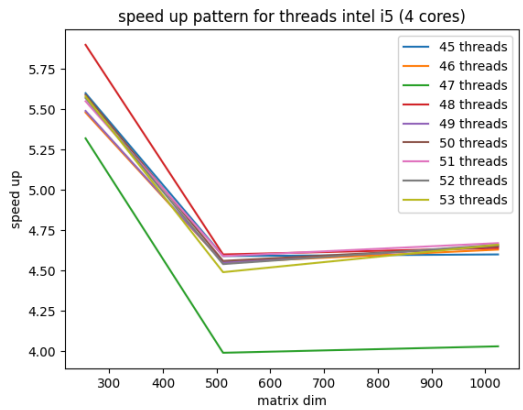












References

1. https://yaikhom.com/2020-05-26-matrix-multiplication.html

2. https://www.cpubenchmark.net/cpu.php?cpu=Apple+M2+8+Core+3500+MHz&id=4922